Roope Aaltonen

Pill Dispenser Project

Embedded Systems Programming Course Project

Metropolia University of Applied Sciences
Bachelor of Engineering
Information and Communications Technology
Embedded Systems Programming Project Report
15 May 2026

Abstract

Author:	Roope Aaltonen
Title:	Pill Dispenser Project
Number of Pages:	13 pages + 0 appendices
Date:	15 May 2026
Degree:	Bachelor of Engineering
Degree Programme:	Information and Communications Technology
Instructor(s):	Embedded Systems Programming course staff

This report describes the design and implementation of a Raspberry Pi Pico based pill dispenser. The system calibrates a stepper-driven dispenser wheel with an opto fork sensor, waits for the user to start the schedule, and then dispenses one pill position every 30 seconds. A piezo sensor is handled with a GPIO interrupt to detect whether a pill has dropped. The project stores operating state in an AT24C256 EEPROM so that a power loss during motor movement can be detected, recovered by reversing to the calibration opening, and completed after reboot. The device also sends status messages through a LoRa-E5 module using AT commands.

The implementation focuses on clear embedded C structure. Interrupt service routines only record events, while application decisions are made in the main loop. The calibration routine measures both edges of the opto opening and centers the dispenser position before the schedule starts. The result is a readable course project implementation that combines motor control, sensors, non-volatile memory and LoRa communication.

Keywords: Raspberry Pi Pico, stepper motor, EEPROM, LoRaWAN, GPIO interrupt, pill dispenser

---

Contents

1 Introduction

The aim of the project is to build an embedded pill dispenser controller with the Raspberry Pi Pico and the second-year embedded systems board. The required behaviour and mechanical arrangement are based on the course project handout (Metropolia UAS, 2026a). The program controls a stepper motor driven wheel, detects a reference position with an opto fork sensor, detects dropped pills with a piezo sensor, stores recovery data to EEPROM and reports important events with a LoRa-E5 module.

The project is implemented in C with the Raspberry Pi Pico SDK and its hardware libraries (Raspberry Pi Ltd., n.d.). The main source file is project/Pill_dispenser/main.c and the build target is pill_dispenser. The run configuration flashes the Pico through OpenOCD and the debug probe, and the monitor configuration shows the debug output through the serial connection.

2 Hardware Connections

The implementation uses the GPIO pins specified for the course board and project hardware (Metropolia UAS, 2026a; Metropolia UAS, 2026b). UART0 is kept for debug output and the LoRa module is connected to UART1.

Function	GPIO pin	Purpose
SW0 button	GP9	User input for calibration and schedule start. Active low.
Status LED	GP20	Blinking wait indication, ready indication and error blinking.
Stepper coils	GP2, GP3, GP6, GP13	Four half-step outputs for the dispenser wheel motor.
LoRa-E5 UART	GP4 / GP5	UART1 at 9600 baud for AT commands.
EEPROM I2C	GP16 / GP17	I2C0 at 100 kHz for the AT24C256 EEPROM.
Piezo sensor	GP27	Falling-edge GPIO interrupt for pill drop detection.
Opto fork sensor	GP28	Active low reference signal for wheel calibration.

3 Program Structure

The code is divided into small functions by hardware responsibility and application logic. This keeps the main loop readable while still making the low-level behaviour easy to inspect.

- EEPROM functions read, validate and save the dispenser state with a checksum.
- GPIO initialization sets up the button, LED, opto fork, piezo interrupt and stepper outputs.
- Stepper functions output the half-step sequence and move the dispenser wheel.
- Calibration functions find the opto opening, measure one full revolution and center the wheel.
- LoRa functions send the required AT commands and short status messages.

- The main loop runs the application state machine and handles user interaction.

The only changing global runtime variable is piezo_edges. It is global because it is shared between the piezo interrupt handler and the main loop. The ISR only increments this counter, so the interrupt logic stays short and predictable.

4 Code Examples

The following excerpts show the most important implementation choices. The full source code remains in project/Pill_dispenser/main.c, but these short examples show how interrupts, persistent recovery and LoRa reporting are handled.

4.1 Minimal interrupt handler

```
static void gpio_irq_callback(uint gpio, uint32_t events) {
    if (gpio == PIEZO_PIN && (events & GPIO_IRQ_EDGE_FALL)) {
        piezo_edges++;
    }
}
```

The interrupt service routine only records that the piezo input saw a falling edge. It does not print text, access EEPROM, move the motor or decide application state. This keeps the interrupt short and leaves the actual pill-detection decision to normal program flow.

4.2 Saved turn state before motor movement

```
state->turn in progress = 1u;
(void)save state(state);
lora_send_status(lora_joined, "turn_start", state->pills_left);

bool detected = move compartment(state, stepper pins, current step);

if (state->pills_left > 0u) {
    state->pills_left--;
}

state->turn_in_progress = 0u;
```

The EEPROM state is saved before the wheel starts moving. If power is lost during this movement, the next boot sees turn_in_progress and knows that the dispense

had already started. The flag is cleared only after the compartment movement has finished.

4.3 Recovery after interrupted movement

```
if (!reverse_to_reference_opening(state, stepper_pins, current_step)) {
    state->mode = MODE_WAIT_CALIBRATION;
    return false;
}

uint32_t completed_dispenses = 7u - state->pills_left;
uint32_t restore_steps = completed_dispenses * steps_per_compartment(state);

if (restore_steps > 0u) {
    move_steps_forward(stepper_pins, current_step, restore_steps);
}
```

The recovery logic does not store every individual motor step. After a reset during movement, the wheel first moves backwards to the known opto reference opening. It then moves forward through already completed positions and completes the interrupted dispense before the normal schedule continues.

4.4 LoRaWAN status message

```
snprintf(message, sizeof(message), "%s left=%u", event, pills_left);
snprintf(command, sizeof(command), "AT+MSG=\"%s\"", message);
send_lora_command(command);
```

LoRa output is created as a short status string containing the event name and the remaining pill count. The string is sent to the LoRa-E5 module with the AT+MSG command. The AppKey is used for setup but is not printed to the debug console.

5 State Machine

The dispenser behaviour is controlled with three saved operating modes.

Mode	Meaning
Wait calibration	Initial state. The LED blinks and the system waits for SW0 before calibration.
Ready to start	Calibration is complete. The LED stays on and the system waits for the second SW0 press.
Running	The dispenser releases one compartment every 30 seconds until the wheel is empty.

The normal operating sequence is intentionally simple:

1. Load saved state from EEPROM after reset.

2. If a motor turn was interrupted, reverse to the calibration opening, restore position and complete the interrupted dispense immediately.

3. Wait for SW0 and calibrate the dispenser wheel.

4. Wait for SW0 again and start the 30 second schedule.

5. Move one compartment at each interval and check the piezo sensor result.

6. After seven positions, return to calibration mode for refilling.

6 Calibration

The opto fork sensor is active low. When the calibration opening reaches the sensor, the input changes from high to low. The program first detects this falling edge and then continues until the signal rises again. This gives the measured width of the opto opening.

After the rising edge, the wheel continues to the next falling edge. The measured opening width plus the distance to the next falling edge gives one full wheel revolution. The result is divided by eight because the dispenser wheel has eight pill positions.

After the revolution measurement, the motor moves half of the opto opening width forward. This centers the dispenser opening on the opto sensor instead of stopping at the first edge of the opening. Impossible measurements are rejected so that a disconnected or incorrectly wired opto fork cannot produce a valid calibration.

7 Dispensing and Piezo Detection

Before a compartment movement starts, the program clears the piezo edge counter. The piezo GPIO interrupt then counts falling edges while the motor turns and during a short fall window after the motor stops.

If at least one piezo falling edge is detected, the dispense is reported as successful. If no edge is detected, the status LED blinks five times. The pill position counter is still decremented after each motor movement because the wheel has physically advanced to the next compartment even if the sensor did not detect a pill.

8 EEPROM Recovery

The AT24C256 EEPROM stores the current mode, pills left, measured steps per revolution, whether a motor turn is in progress and a checksum. The EEPROM is used through its two-wire serial interface and page-write limitations are respected according to the datasheet (Atmel Corporation, 2007). The state is saved before a turn starts and after the turn finishes, so the EEPROM is not written on every motor step.

If reset happens while the wheel is moving, the next boot detects the interrupted turn. The recovery routine moves the wheel backwards until the calibration opening is found, centers on that opening, and then moves forward through already emptied positions to return to the last completed position. After that, the interrupted dispense is completed immediately before the normal schedule continues. This avoids saving every motor step to EEPROM while still giving the dose that had already started.

9 LoRaWAN Communication

The LoRa-E5 module is configured with AT commands for OTAA mode, Class A operation, port 8 and DR5. The AT command format follows the LoRa-E5 command specification (Seeed Technology Co., Ltd., 2020). The AppKey comes from the course device list, but the program does not print the key to the debug console.

After joining the network, the application sends short status messages such as boot, calibration_start, calibrated, started, turn_start, dispensed, no_pill and empty. Each status message also includes the number of pill positions left. Motor

control continues even if LoRa is unavailable, because dispensing must not depend on network availability.

10 Building, Running and Testing

The project is built with the Pico SDK CMake target pill_dispenser:

cmake --build cmake-build-debug --target pill_dispenser -j 6

In CLion, the Run pill_dispenser + Monitor configuration flashes the target through the debug probe and opens the serial monitor. The monitor is useful for checking calibration, LoRa joining, schedule start and dispense events.

Recommended tests are:

1. Build and flash the pill_dispenser target through the debugger.
2. Open the monitor and confirm that the startup text appears.
3. Press SW0 and confirm that calibration starts and reports measured half steps.
4. Check that calibration centers the wheel on the opto opening.
5. Press SW0 again and confirm that the 30 second schedule starts.
6. Check that the stepper moves one compartment at every interval.
7. Test both pill detected and no-pill cases with the piezo sensor.
8. Reset during a motor turn and verify that the wheel reverses to the calibration opening, restores position and completes the interrupted dispense.

9. Test LoRaWAN status reception in the school network with the provided receiver script.

11 Practical Test Results

Functional testing was carried out with the assembled course board and LoRaWAN coverage available at school. The LoRaWAN part could not be repeated later at home, because joining the network depends on the campus gateway and course network coverage. Therefore, the report separates the classroom observations from the home limitation instead of treating a missing home join as a software failure.

The serial monitor confirmed the LoRa-E5 setup sequence. The module answered the basic AT check with LoRa: +AT: OK, accepted OTAA configuration, joined the network and returned join status lines including +JOIN: Network joined and +JOIN: Done. After joining, the program sent short AT+MSG status messages for boot, calibration, schedule start and dispensing events. This verified that the code produces LoRa output through the LoRa-E5 module, while still allowing the dispenser to continue if the network is temporarily unavailable.

Calibration was tested with SW0. The wheel first searched for the opto fork reference opening, then measured one full revolution. In the working test, the measured value was approximately 4097 half steps per revolution, which gives 512 half steps per compartment for the eight-position dispenser wheel. The final version does not stop at the first opto edge; it moves to the middle of the measured opening so that the dispensing hole is aligned more accurately.

The dispensing schedule was tested after calibration by pressing SW0 again. The monitor showed the schedule start message and then one compartment movement every 30 seconds. During the test without a reliable pill drop on every movement, the no-pill path was also seen with output such as No pill detected, pills left: 6. The program reduced the remaining position count after the mechanical movement, reported the event through LoRa, and used the LED

warning behavior for the missing piezo detection. This was useful because it verified both the normal motor path and the sensor-failure path.

EEPROM recovery was checked by restarting the device and confirming that the program restored the saved mode and pill count instead of always starting from an empty default state. The code saves turn_in_progress before the motor starts and clears it only after the turn is complete. If power is lost during a movement, the next boot uses this flag to reverse back to the opto reference, return to the last completed position and finish the interrupted dispense immediately. This avoids writing every motor step to EEPROM while still preserving the started dose.

The main practical observation from testing was mechanical alignment. A small visible offset can occur if the wheel, opto fork and dispenser hole are not physically mounted in exactly the same reference position. The software compensates for this by measuring the opto opening width and moving to its midpoint, but final accuracy still depends on the physical assembly.

12 Conclusion

The finished project combines the central embedded systems topics from the course: GPIO input and output, interrupts, UART communication, I2C EEPROM access, stepper motor control, non-blocking application flow and persistent recovery state. The code keeps the interrupt handler minimal and places application decisions in normal program flow, which makes the system easier to debug and maintain.

The most important functional limitation is mechanical alignment. The software now centers the wheel using the measured opto opening, but a small physical offset may still be needed if the dispenser hole and sensor position are not perfectly aligned in the assembled mechanism.

13 References

Atmel Corporation. 2007. AT24C128/256: Two-wire Serial EEPROMs. Datasheet 0670T-SEEPR-3/07.

Metropolia University of Applied Sciences. 2026a. Pill dispenser. Embedded Systems Programming course project handout. Course material.

Metropolia University of Applied Sciences. 2026b. RPI second year board. Course hardware documentation. Course material.

Raspberry Pi Ltd. n.d. Raspberry Pi Pico C/C++ SDK documentation. Available at: https://www.raspberrypi.com/documentation/pico-sdk/ (Accessed 15 May 2026).

Seeed Technology Co., Ltd. 2020. LoRa Wireless Module - Powered by STM32WLE5: AT Command Specification. Version V1.0.